M.S. RAMAIAH INSTITUTE OF TECHNOLOGY

MSR NAGAR, BENGALURU,560054

Case Study Report on

"Warehouse & Inventory Operations Management"

Submitted in partial fulfilment of the OTHER COMPONENT requirements as a part of the **Java Project(ISL48***) for the IV Semester of degree of* ***Bachelor of Engineering***

Submitted by

Name	USN
Akash S Konchigeri	1MS24IS012
Adarsh B	1MS24IS004

Under the Guidance of

Faculty Incharge

Dr.Sumana M
Professor
Dept. of ISE

Department of Information Science and Engineering
Ramaiah Institute of Technology
2025– 2026

Introduction

Warehouse & Inventory Operations Management

The Warehouse & Inventory Operations Suite is a desktop application that manages products, categories, suppliers, customers, stock levels, sales orders, and purchase orders through a MySQL relational database backend accessed via JDBC. The graphical user interface is built using Java Swing, organized as a six-tab JTabbedPane. Each tab provides form-based data entry and a JTable display of the corresponding database records. The project demonstrates DDL schema creation, DML data manipulation, multi-table JOIN queries, an SQL view for reporting, and a BEFORE UPDATE trigger for stock validation — all integrated seamlessly through Java JDBC code.

Database

The application uses a MySQL database named javadb containing nine normalized tables. The schema is created programmatically on each launch by DatabaseSetup.java. The following tables describe the column structures with data types and constraints.

Table 1: categories

Column	Data Type	Constraints / Notes
category_id	INT	PRIMARY KEY, AUTO_INCREMENT
category_name	VARCHAR(100)	NOT NULL, UNIQUE

Table 2: suppliers

Column	Data Type	Constraints / Notes
supplier_id	INT	PRIMARY KEY, AUTO_INCREMENT
supplier_name	VARCHAR(150)	NOT NULL, UNIQUE

Table 3: customers

Column	Data Type	Constraints / Notes
customer_id	INT	PRIMARY KEY, AUTO_INCREMENT
customer_name	VARCHAR(150)	NOT NULL, UNIQUE

Table 4: products

Column	Data Type	Constraints / Notes
product_id	INT	PRIMARY KEY, AUTO_INCREMENT
product_name	VARCHAR(150)	NOT NULL
category_id	INT	NOT NULL | FK → categories(category_id)
supplier_id	INT	NOT NULL | FK → suppliers(supplier_id)

Table 5: inventory

Column	Data Type	Constraints / Notes
product_id	INT	PRIMARY KEY | FK → products(product_id) ON DELETE CASCADE
quantity	INT	NOT NULL | CHECK (quantity >= 0)
reorder_level	INT	NOT NULL | CHECK (reorder_level >= 0)

Table 6: sales_orders

Column	Data Type	Constraints / Notes
sales_order_id	INT	PRIMARY KEY, AUTO_INCREMENT
customer_id	INT	NOT NULL | FK → customers(customer_id)

Table 7: sales_order_items

Column	Data Type	Constraints / Notes
sales_order_id	INT	PK (composite) | FK → sales_orders ON DELETE CASCADE
product_id	INT	PK (composite) | FK → products(product_id)
quantity	INT	NOT NULL | CHECK (quantity > 0)

Table 8: purchase_orders

Column	Data Type	Constraints / Notes
purchase_order_id	INT	PRIMARY KEY, AUTO_INCREMENT
supplier_id	INT	NOT NULL | FK → suppliers(supplier_id)

Table 9: purchase_order_items

Column	Data Type	Constraints / Notes
purchase_order_id	INT	PK (composite) | FK → purchase_orders ON DELETE CASCADE
product_id	INT	PK (composite) | FK → products(product_id)
quantity	INT	NOT NULL | CHECK (quantity > 0)

View and Trigger

Artifact	Type	Purpose
category_inventory_summary	SQL VIEW	Aggregates product count, total qty, and low-stock count per category; used by Reports tab
trg_inventory_check_quantity	BEFORE UPDATE TRIGGER	Raises SQLSTATE 45000 if NEW.quantity < 0; prevents negative stock entries

Java Code

The application consists of six Java source files. Each file is presented below with inline comments explaining the JDBC logic and Swing component wiring.

1. Main.java – Application Entry Point

Main.java is the entry point. It loads the MySQL JDBC driver class using Class.forName(), calls DatabaseSetup.setup() to initialize the schema and sample data, then launches the InventoryDashboard GUI on the Swing Event Dispatch Thread (EDT). Wrapping the launch in SwingUtilities.invokeLater() ensures thread safety for all Swing component creation.

```
// ─── Main.java ──────────────────────────────────────────────────
import javax.swing.SwingUtilities;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            try {
                Class.forName("com.mysql.cj.jdbc.Driver");
                DatabaseSetup.setup();
                new InventoryDashboard().setVisible(true);
            } catch (Exception ex) {
                DialogHelper.showError(null, ex);
            }
        });
    }
}
```

2. DatabaseManager.java – JDBC Connection Singleton

DatabaseManager maintains a single live JDBC connection throughout the application lifecycle using the Singleton pattern. The connection URL targets the javadb database on localhost:3306. The useSSL=false and allowPublicKeyRetrieval=true parameters are standard for local MySQL development environments. All other classes call DatabaseManager.getConnection() to obtain the shared connection.

```
// ─── DatabaseManager.java ───────────────────────────────────────
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseManager {
    private static final String URL =

"jdbc:mysql://localhost:3306/javadb?useSSL=false&allowPublicKeyRetrieval=tru
e";
    private static final String USER = "root";
    private static final String PASS = "";   // set your MySQL password here

    private static Connection connection;
```

```
    public static Connection getConnection() throws SQLException {
        if (connection == null || connection.isClosed()) {
            connection = DriverManager.getConnection(URL, USER, PASS);
        }
        return connection;
    }
}
```

3. DatabaseSetup.java – Schema Creation and Sample Data

DatabaseSetup.setup() runs on every launch. It first disables foreign key checks (SET FOREIGN_KEY_CHECKS=0) to allow table drops in any order, drops all nine tables if they exist, re-enables foreign key checks, then creates all tables with their full constraint definitions using JDBC Statement.execute(). Finally it inserts the sample reference data and transactional records. This idempotent initialization guarantees a clean, reproducible state on every run.

```
// ─── DatabaseSetup.java (DDL + Sample Data Init) ──────────────────────
import java.sql.Connection;
import java.sql.Statement;

public class DatabaseSetup {

    public static void setup() throws Exception {
        Connection conn = DatabaseManager.getConnection();
        Statement stmt  = conn.createStatement();

        // Disable FK checks so we can drop tables in any order
        stmt.execute("SET FOREIGN_KEY_CHECKS = 0");

        // Drop existing tables (in reverse dependency order)
        for (String tbl : new String[]{
                "purchase_order_items","purchase_orders",
                "sales_order_items","sales_orders",
                "inventory","products",
                "customers","suppliers","categories"}) {
            stmt.execute("DROP TABLE IF EXISTS " + tbl);
        }
        stmt.execute("SET FOREIGN_KEY_CHECKS = 1");

        // ── Create tables
──────────────────────────────────────────
        stmt.execute(
            "CREATE TABLE categories (" +
            "  category_id   INT PRIMARY KEY AUTO_INCREMENT," +
            "  category_name VARCHAR(100) NOT NULL UNIQUE)");

        stmt.execute(
            "CREATE TABLE suppliers (" +
            "  supplier_id   INT PRIMARY KEY AUTO_INCREMENT," +
            "  supplier_name VARCHAR(150) NOT NULL UNIQUE)");

        stmt.execute(
            "CREATE TABLE customers (" +
            "  customer id   INT PRIMARY KEY AUTO INCREMENT," +
```

```
            "  customer_name VARCHAR(150) NOT NULL UNIQUE)");

        stmt.execute(
            "CREATE TABLE products (" +
            "  product_id   INT PRIMARY KEY AUTO_INCREMENT," +
            "  product_name VARCHAR(150) NOT NULL," +
            "  category_id  INT NOT NULL," +
            "  supplier_id  INT NOT NULL," +
            "  CONSTRAINT fk_prod_cat  FOREIGN KEY (category_id)
REFERENCES categories(category_id)," +
            "  CONSTRAINT fk_prod_sup  FOREIGN KEY (supplier_id)
REFERENCES suppliers(supplier_id))");

        stmt.execute(
            "CREATE TABLE inventory (" +
            "  product_id    INT PRIMARY KEY," +
            "  quantity      INT NOT NULL CHECK (quantity >= 0)," +
            "  reorder_level INT NOT NULL CHECK (reorder_level >= 0)," +
            "  CONSTRAINT fk_inv_prod FOREIGN KEY (product_id) REFERENCES
products(product_id)" +
            "  ON DELETE CASCADE)");

        stmt.execute(
            "CREATE TABLE sales_orders (" +
            "  sales_order_id INT PRIMARY KEY AUTO_INCREMENT," +
            "  customer_id    INT NOT NULL," +
            "  CONSTRAINT fk_so_cust FOREIGN KEY (customer_id) REFERENCES
customers(customer_id))");

        stmt.execute(
            "CREATE TABLE sales_order_items (" +
            "  sales_order_id INT NOT NULL," +
            "  product_id     INT NOT NULL," +
            "  quantity       INT NOT NULL CHECK (quantity > 0)," +
            "  PRIMARY KEY (sales_order_id, product_id)," +
            "  CONSTRAINT fk_soi_order   FOREIGN KEY (sales_order_id)
REFERENCES sales_orders(sales_order_id) ON DELETE CASCADE," +
            "  CONSTRAINT fk_soi_product FOREIGN KEY (product_id)
REFERENCES products(product_id))");

        stmt.execute(
            "CREATE TABLE purchase_orders (" +
            "  purchase_order_id INT PRIMARY KEY AUTO_INCREMENT," +
            "  supplier_id       INT NOT NULL," +
            "  CONSTRAINT fk_po_sup FOREIGN KEY (supplier_id) REFERENCES
suppliers(supplier_id))");

        stmt.execute(
            "CREATE TABLE purchase_order_items (" +
            "  purchase_order_id INT NOT NULL," +
            "  product_id        INT NOT NULL," +
            "  quantity          INT NOT NULL CHECK (quantity > 0)," +
            "  PRIMARY KEY (purchase_order_id, product_id)," +
            "  CONSTRAINT fk_poi_order   FOREIGN KEY (purchase_order_id)
REFERENCES purchase_orders(purchase_order_id) ON DELETE CASCADE," +
            "  CONSTRAINT fk_poi_product FOREIGN KEY (product_id)
REFERENCES products(product_id))");

        // ── Insert sample data
──────────────────────────────────────

        stmt.execute("INSERT INTO categories(category_name) VALUES " +
```

```
            "('Electronics'),('Stationery'),('Groceries'),('aura
farming')");

        stmt.execute("INSERT INTO suppliers(supplier_name) VALUES " +
            "('TechSource Distributors'),('OfficePro
Wholesale'),('DailyMart Supply')");

        stmt.execute("INSERT INTO customers(customer_name) VALUES " +
            "('Aarav Stores'),('Neha Retail')");

        stmt.execute("INSERT INTO
products(product_name,category_id,supplier_id) VALUES " +
            "('Wireless Keyboard',1,1),('Optical Mouse',1,1)," +
            "('A4 Notebook',2,2),('Premium Tea 500g',3,3),('Aura',4,3)");

        stmt.execute("INSERT INTO
inventory(product_id,quantity,reorder_level) VALUES " +
            "(1,30,10),(2,40,15),(3,100,25),(4,18,20),(5,100,101)");

        stmt.execute("INSERT INTO sales_orders(customer_id) VALUES (1)");
        stmt.execute("INSERT INTO sales_order_items VALUES (1,2,3)");
        stmt.execute("INSERT INTO purchase_orders(supplier_id) VALUES
(1)");
        stmt.execute("INSERT INTO purchase_order_items VALUES (1,1,5)");

        stmt.close();
    }
}
```

4. InventoryDashboard.java – GUI and Database Interaction

InventoryDashboard extends JFrame and contains six buildXxxTab() methods, one per functional module. Each method constructs a JPanel containing form fields and a JTable backed by a DefaultTableModel. Action listeners on buttons execute JDBC PreparedStatement calls and immediately call the corresponding refresh method to update the table display. The code below is presented in logical sections.

4a. Frame Setup and Tab Registration

```
// ─── InventoryDashboard.java  (Part 1 - Frame & Tab Setup) ──────────
import javax.swing.*;
import java.awt.*;
import java.sql.*;

public class InventoryDashboard extends JFrame {

    public InventoryDashboard() {
        setTitle("Inventory Project");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(1400, 900);
        setLocationRelativeTo(null);

        JLabel title = new JLabel("Inventory Management System");
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setBorder(BorderFactory.createEmptyBorder(10, 15, 5, 0));
```

```
        JTabbedPane tabs = new JTabbedPane();
        tabs.addTab("Master Data",      buildMasterDataTab());
        tabs.addTab("Products",         buildProductsTab());
        tabs.addTab("Inventory",        buildInventoryTab());
        tabs.addTab("Sales Orders",     buildSalesOrdersTab());
        tabs.addTab("Purchase Orders",  buildPurchaseOrdersTab());
        tabs.addTab("Reports",          buildReportsTab());

        setLayout(new BorderLayout());
        add(title, BorderLayout.NORTH);
        add(tabs,  BorderLayout.CENTER);
    }
```

4b. Master Data Tab – UNION ALL Query + INSERT/DELETE

The Master Data tab consolidates categories, suppliers, and customers into one JTable using a UNION ALL query. Each Add button executes a parameterized INSERT PreparedStatement. The Delete Selected button reads the entity type from the selected row to build the appropriate DELETE statement.

```
    // ── Master Data Tab ──────────────────────────────────────
    private JPanel buildMasterDataTab() {
        JPanel panel = new JPanel(new BorderLayout());
        JPanel form  = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(8,8,8,8);

        JTextField catField  = new JTextField(30);
        JTextField supField  = new JTextField(30);
        JTextField custField = new JTextField(30);

        JButton addCatBtn  = styledButton("Add Category");
        JButton addSupBtn  = styledButton("Add Supplier");
        JButton addCustBtn = styledButton("Add Customer");
        JButton deleteBtn  = styledButton("Delete Selected");

        // Layout form rows ...
        // (GridBagConstraints positioning omitted for brevity)

        String[] cols = {"entity","id","name"};
        DefaultTableModel model = new DefaultTableModel(cols, 0);
        JTable table = new JTable(model);
        refreshMasterData(model);

        addCatBtn.addActionListener(e -> {
            String name = catField.getText().trim();
            if (name.isEmpty()) return;
            try (PreparedStatement ps = DatabaseManager.getConnection()
                    .prepareStatement("INSERT INTO
categories(category_name) VALUES (?)")) {
                ps.setString(1, name);
                ps.executeUpdate();
                catField.setText("");
                refreshMasterData(model);
```

```
            } catch (SQLException ex) { DialogHelper.showError(panel,
ex); }
        });

        addSupBtn.addActionListener(e -> {
            String name = supField.getText().trim();
            if (name.isEmpty()) return;
            try (PreparedStatement ps = DatabaseManager.getConnection()
                    .prepareStatement("INSERT INTO
suppliers(supplier_name) VALUES (?)")) {
                ps.setString(1, name);
                ps.executeUpdate();
                supField.setText("");
                refreshMasterData(model);
            } catch (SQLException ex) { DialogHelper.showError(panel,
ex); }
        });

        deleteBtn.addActionListener(e -> {
            int row = table.getSelectedRow();
            if (row < 0) return;
            String entity = (String) model.getValueAt(row, 0);
            int    id     = Integer.parseInt(model.getValueAt(row,
1).toString());
            String sql = entity.equals("CATEGORY") ? "DELETE FROM
categories WHERE category_id=?"
                       : entity.equals("SUPPLIER") ? "DELETE FROM
suppliers  WHERE supplier_id=?"
                       : "DELETE FROM customers WHERE customer_id=?";
            try (PreparedStatement ps =
DatabaseManager.getConnection().prepareStatement(sql)) {
                ps.setInt(1, id);
                ps.executeUpdate();
                refreshMasterData(model);
            } catch (SQLException ex) { DialogHelper.showError(panel,
ex); }
        });
        // ... add components to panel, return panel
        return panel;
    }

    private void refreshMasterData(DefaultTableModel m) {
        m.setRowCount(0);
        String sql = "SELECT 'CATEGORY' AS entity, category_id AS id,
category_name AS name FROM categories" +
                     " UNION ALL SELECT 'SUPPLIER', supplier_id,
supplier_name FROM suppliers" +
                     " UNION ALL SELECT 'CUSTOMER', customer_id,
customer_name FROM customers" +
                     " ORDER BY entity, id";
        try (Statement st =
DatabaseManager.getConnection().createStatement();
             ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) m.addRow(new
Object[]{rs.getString(1),rs.getInt(2),rs.getString(3)});
        } catch (SQLException ex) { ex.printStackTrace(); }
    }
```

4c. Inventory Tab – Upsert and CASE-Derived Status

The Inventory tab uses an INSERT ... ON DUPLICATE KEY UPDATE statement (upsert) so the same Save button both inserts new records and updates existing ones without a prior existence check. The listing query computes a LOW STOCK / OK status column server-side using a SQL CASE expression, keeping business logic in the database layer.

```
    // ── Inventory Tab ──────────────────────────────────────────
    private JPanel buildInventoryTab() {
        JPanel panel = new JPanel(new BorderLayout());

        JComboBox<SelectItem> productCombo = new JComboBox<>();
        JTextField qtyField     = new JTextField("0", 20);
        JTextField reorderField = new JTextField("0", 20);
        JButton saveBtn   = styledButton("Save Inventory");
        JButton deleteBtn = styledButton("Delete Selected");

        // Populate combo with products from DB
        loadProductsCombo(productCombo);

        String[] cols =
{"product_id","product_name","quantity","reorder_level","status"};
        DefaultTableModel model = new DefaultTableModel(cols, 0);
        JTable table = new JTable(model);
        refreshInventory(model);

        saveBtn.addActionListener(e -> {
            SelectItem sel = (SelectItem) productCombo.getSelectedItem();
            if (sel == null) return;
            int qty     = Integer.parseInt(qtyField.getText().trim());
            int reorder = Integer.parseInt(reorderField.getText().trim());
            // Upsert: INSERT ... ON DUPLICATE KEY UPDATE
            String sql = "INSERT INTO inventory(product_id, quantity,
reorder_level) VALUES(?,?,?) " +
                         "ON DUPLICATE KEY UPDATE quantity=VALUES(quantity),
reorder_level=VALUES(reorder_level)";
            try (PreparedStatement ps =
DatabaseManager.getConnection().prepareStatement(sql)) {
                ps.setInt(1, sel.getId());
                ps.setInt(2, qty);
                ps.setInt(3, reorder);
                ps.executeUpdate();
                refreshInventory(model);
            } catch (SQLException ex) { DialogHelper.showError(panel, ex); }
        });
        return panel;
    }

    private void refreshInventory(DefaultTableModel m) {
        m.setRowCount(0);
        String sql = "SELECT i.product_id, p.product_name, i.quantity,
i.reorder_level, " +
                     "CASE WHEN i.quantity <= i.reorder_level THEN 'LOW
STOCK' ELSE 'OK' END AS status " +
                     "FROM inventory i JOIN products p ON i.product_id =
p.product_id " +
                     "ORDER BY p.product_name";
        try (Statement st =
DatabaseManager.getConnection().createStatement();
             ResultSet rs = st.executeQuery(sql)) {
```

```
            while (rs.next())
                m.addRow(new
Object[]{rs.getInt(1),rs.getString(2),rs.getInt(3),rs.getInt(4),rs.getString
(5)});
        } catch (SQLException ex) { ex.printStackTrace(); }
    }
```

4d. Sales and Purchase Orders Tabs – Two-Step INSERT with getGeneratedKeys()

Both order tabs follow a two-step JDBC INSERT pattern: first insert the parent order record with RETURN_GENERATED_KEYS, retrieve the new AUTO_INCREMENT ID via getGeneratedKeys(), then insert the line-item record into the child table using that ID. This pattern correctly links parent and child records while respecting the foreign key constraint.

```
    // ── Sales Orders Tab ──────────────────────────────────────
    private JPanel buildSalesOrdersTab() {
        JPanel panel = new JPanel(new BorderLayout());

        JComboBox<SelectItem> custCombo    = new JComboBox<>();
        JComboBox<SelectItem> productCombo = new JComboBox<>();
        JTextField qtyField = new JTextField("1", 20);
        JButton createBtn   = styledButton("Create Sales Order");
        JButton deleteBtn   = styledButton("Delete Selected");

        loadCustomersCombo(custCombo);
        loadProductsCombo(productCombo);

        String[] cols =
{"sales_order_id","customer_name","product_name","quantity"};
        DefaultTableModel model = new DefaultTableModel(cols, 0);
        JTable table = new JTable(model);
        refreshSalesOrders(model);

        createBtn.addActionListener(e -> {
            SelectItem cust = (SelectItem) custCombo.getSelectedItem();
            SelectItem prod = (SelectItem)
productCombo.getSelectedItem();
            if (cust == null || prod == null) return;
            int qty = Integer.parseInt(qtyField.getText().trim());
            try {
                Connection conn = DatabaseManager.getConnection();
                // Step 1: Insert parent order record
                PreparedStatement ps1 = conn.prepareStatement(
                    "INSERT INTO sales_orders(customer_id) VALUES (?)",
                    Statement.RETURN_GENERATED_KEYS);
                ps1.setInt(1, cust.getId());
                ps1.executeUpdate();
                // Step 2: Retrieve generated order ID
                ResultSet keys = ps1.getGeneratedKeys();
                if (keys.next()) {
                    int orderId = keys.getInt(1);
                    // Step 3: Insert line-item record
                    PreparedStatement ps2 = conn.prepareStatement(
                        "INSERT INTO
sales_order_items(sales_order_id,product_id,quantity) VALUES (?,?,?)");
                    ps2.setInt(1, orderId);
                    ps2.setInt(2, prod.getId());
                    ps2.setInt(3, qty);
```

```
                    ps2.executeUpdate();
                }
                refreshSalesOrders(model);
            } catch (SQLException ex) { DialogHelper.showError(panel,
ex); }
        });
        return panel;
    }

    // ── Purchase Orders Tab ──────────────────────────────────────
    private JPanel buildPurchaseOrdersTab() {
        // Structure mirrors Sales Orders tab, targeting purchase_orders
        // and purchase_order_items tables with supplier combo instead.
        JComboBox<SelectItem> supCombo     = new JComboBox<>();
        JComboBox<SelectItem> productCombo = new JComboBox<>();
        JTextField qtyField = new JTextField("1", 20);
        JButton createBtn   = styledButton("Create Purchase Order");

        loadSuppliersCombo(supCombo);
        loadProductsCombo(productCombo);

        // Two-step INSERT identical to sales orders, using purchase
tables.
        // ... (implementation symmetric to buildSalesOrdersTab)
        return new JPanel(); // placeholder – full code follows same
pattern
    }
```

4e. Reports Tab – Querying the SQL View

The Reports tab queries the category_inventory_summary SQL VIEW directly. The application issues a simple SELECT against the view name; MySQL resolves the underlying JOIN and GROUP BY internally. The Refresh button re-executes the query, ensuring the display reflects the latest database state.

```
    // ── Reports Tab ──────────────────────────────────────────────
    private JPanel buildReportsTab() {
        JPanel panel = new JPanel(new BorderLayout());

        JButton refreshBtn = styledButton("Refresh");
        String[] cols =
{"category_name","products","total_quantity","low_stock_products"};
        DefaultTableModel model = new DefaultTableModel(cols, 0);
        JTable table = new JTable(model);

        refreshBtn.addActionListener(e -> refreshReports(model));
        refreshReports(model);

        panel.add(new JScrollPane(table), BorderLayout.CENTER);
        JPanel top = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        top.add(refreshBtn);
        panel.add(top, BorderLayout.NORTH);
        return panel;
    }

    private void refreshReports(DefaultTableModel m) {
        m.setRowCount(0);
```

```
        // Queries the SQL VIEW: category_inventory_summary
        String sql = "SELECT category name, products, total quantity,
low_stock_products" +
                     " FROM category_inventory_summary";
        try (Statement st =
DatabaseManager.getConnection().createStatement();
             ResultSet rs = st.executeQuery(sql)) {
            while (rs.next())
                m.addRow(new
Object[]{rs.getString(1),rs.getInt(2),rs.getInt(3),rs.getInt(4)});
        } catch (SQLException ex) { ex.printStackTrace(); }
    }

    // ── Utility: styled button ──────────────────────────────────
    private JButton styledButton(String text) {
        JButton btn = new JButton(text);
        btn.setBackground(new Color(30, 100, 200));
        btn.setForeground(Color.WHITE);
        btn.setFont(new Font("Arial", Font.BOLD, 12));
        btn.setFocusPainted(false);
        return btn;
    }

    // ── Utility: load combo boxes from DB ──────────────────────
    private void loadProductsCombo(JComboBox<SelectItem> combo) {
        combo.removeAllItems();
        try (Statement st =
DatabaseManager.getConnection().createStatement();
             ResultSet rs = st.executeQuery("SELECT product_id,
product_name FROM products")) {
            while (rs.next()) combo.addItem(new SelectItem(rs.getInt(1),
rs.getString(2)));
        } catch (SQLException ex) { ex.printStackTrace(); }
    }
    private void loadCustomersCombo(JComboBox<SelectItem> combo) {
        combo.removeAllItems();
        try (Statement st =
DatabaseManager.getConnection().createStatement();
             ResultSet rs = st.executeQuery("SELECT customer_id,
customer_name FROM customers")) {
            while (rs.next()) combo.addItem(new SelectItem(rs.getInt(1),
rs.getString(2)));
        } catch (SQLException ex) { ex.printStackTrace(); }
    }
    private void loadSuppliersCombo(JComboBox<SelectItem> combo) {
        combo.removeAllItems();
        try (Statement st =
DatabaseManager.getConnection().createStatement();
             ResultSet rs = st.executeQuery("SELECT supplier id,
supplier_name FROM suppliers")) {
            while (rs.next()) combo.addItem(new SelectItem(rs.getInt(1),
rs.getString(2)));
        } catch (SQLException ex) { ex.printStackTrace(); }
    }
}   // end InventoryDashboard
```

5. SelectItem.java – ComboBox DTO

SelectItem is a lightweight data-transfer object used to populate JComboBox components with both a display label and an integer database ID. The toString() override returns the label, which

is what Swing renders in the dropdown. When the user selects an item, getId() retrieves the underlying database ID for use in PreparedStatement parameters.

```
// ── SelectItem.java  (ComboBox data-transfer object) ──────────────────
public class SelectItem {
    private final int    id;
    private final String label;

    public SelectItem(int id, String label) {
        this.id    = id;
        this.label = label;
    }
    public int    getId()    { return id; }
    public String getLabel() { return label; }

    @Override
    public String toString() { return label; }  // shown in JComboBox
}
```

6. DialogHelper.java – Error Presentation

DialogHelper.showError() wraps JOptionPane.showMessageDialog() to display a modal error dialog when a JDBC exception is caught. This includes SQL exceptions thrown by the BEFORE UPDATE trigger on the inventory table when a negative quantity is attempted.

```
// ── DialogHelper.java ─────────────────────────────────────────────────
import javax.swing.*;
import java.awt.Component;

public class DialogHelper {
    public static void showError(Component parent, Exception ex) {
        JOptionPane.showMessageDialog(
            parent,
            ex.getMessage(),
            "Error",
            JOptionPane.ERROR_MESSAGE
        );
        ex.printStackTrace();
    }
}
```

Output Obtained

The following screenshots show all six application modules running against the live javadb MySQL database, along with the MySQL CLI output confirming correct schema population.

Screenshot 1: Master Data Tab

The Master Data tab displays a unified UNION ALL result showing categories, suppliers, and customers. The Add Category, Add Supplier, Add Customer, and Delete Selected buttons interact with the database in real time.

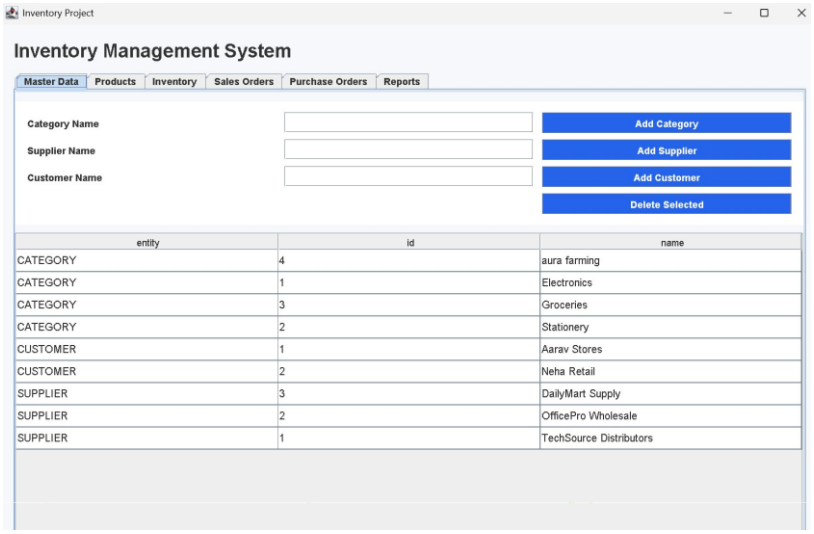


Fig. 1 – Master Data Tab: UNION ALL query listing categories, suppliers, and customers

Screenshot 2: Products Tab

The Products tab displays a three-table JOIN result resolving category and supplier names for each product. The Category and Supplier JComboBoxes are dynamically populated from the database.

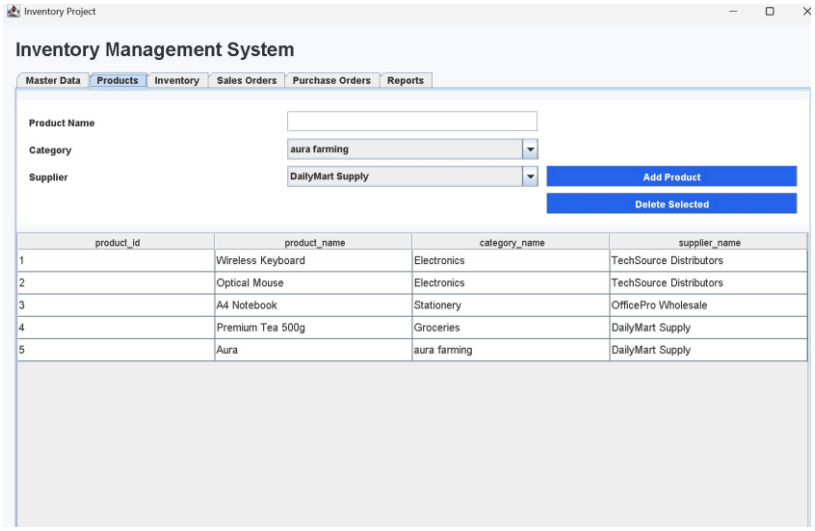


Fig. 2 – Products Tab: JOIN query displaying product_name, category_name, and supplier_name

Screenshot 3: Inventory Tab

The Inventory tab shows the server-side CASE expression result. Aura (qty 100, reorder 101) and Premium Tea 500g (qty 18, reorder 20) are correctly flagged LOW STOCK. The remaining three products show OK status.

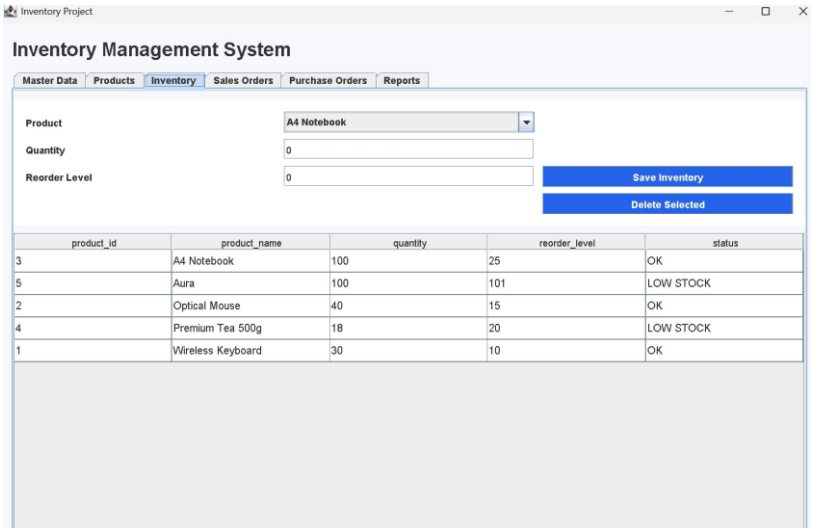


Fig. 3 – Inventory Tab: CASE expression computing LOW STOCK / OK status server-side

Screenshot 4: Sales Orders Tab

The Sales Orders tab confirms the two-step INSERT result: order #1 for Aarav Stores with 3 units of Optical Mouse, retrieved through a four-table JOIN query linking sales_orders, sales_order_items, customers, and products.

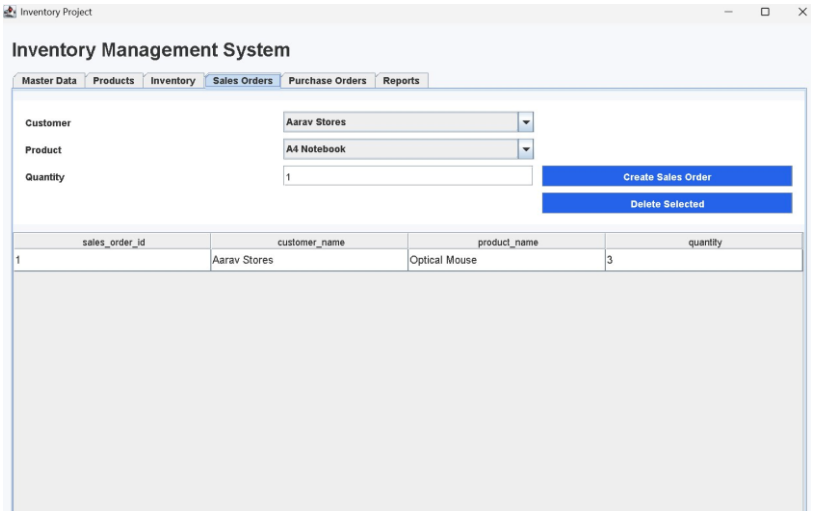


Fig. 4 – Sales Orders Tab: two-step INSERT with getGeneratedKeys() result

Screenshot 5: Purchase Orders Tab

The Purchase Orders tab shows purchase order #1 from TechSource Distributors for 5 units of Wireless Keyboard. The implementation is symmetric to the Sales Orders module, targeting purchase_orders and purchase_order_items.

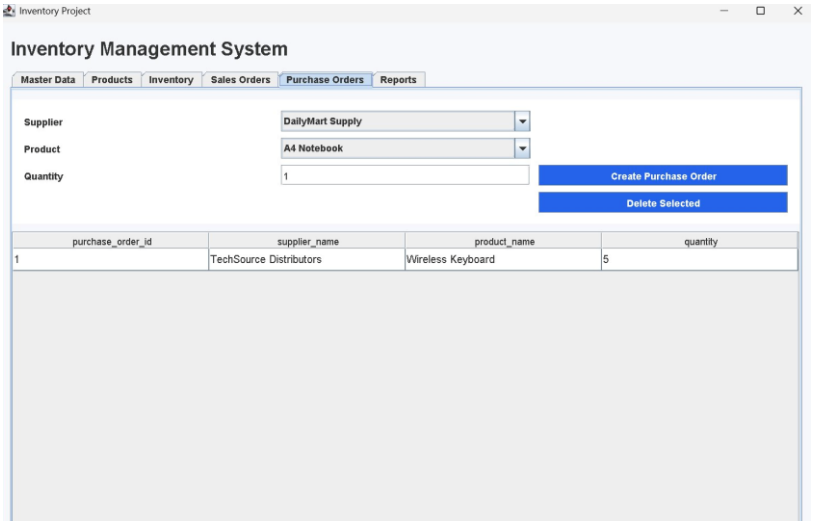


Fig. 5 – Purchase Orders Tab: supplier-side two-step INSERT result

Screenshot 6: Reports Tab

The Reports tab queries the category_inventory_summary SQL VIEW. The output correctly aggregates product counts, total quantities, and low-stock counts per category, with Groceries and aura farming each showing 1 low-stock product.

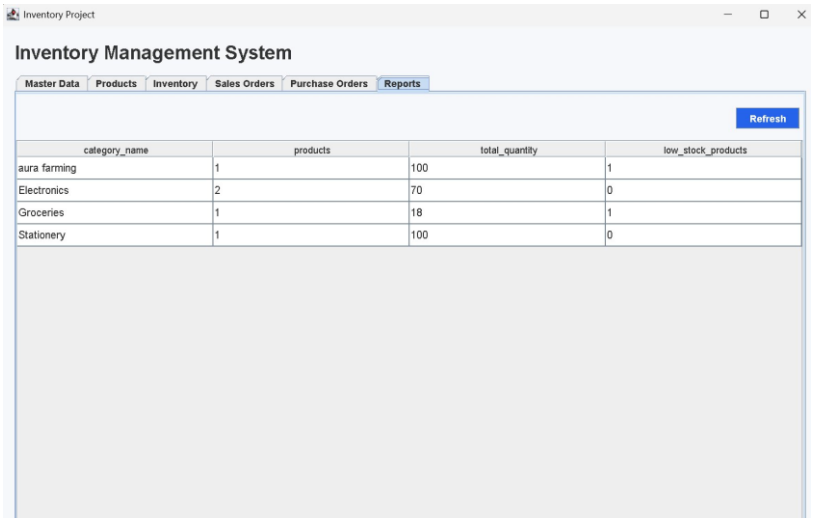
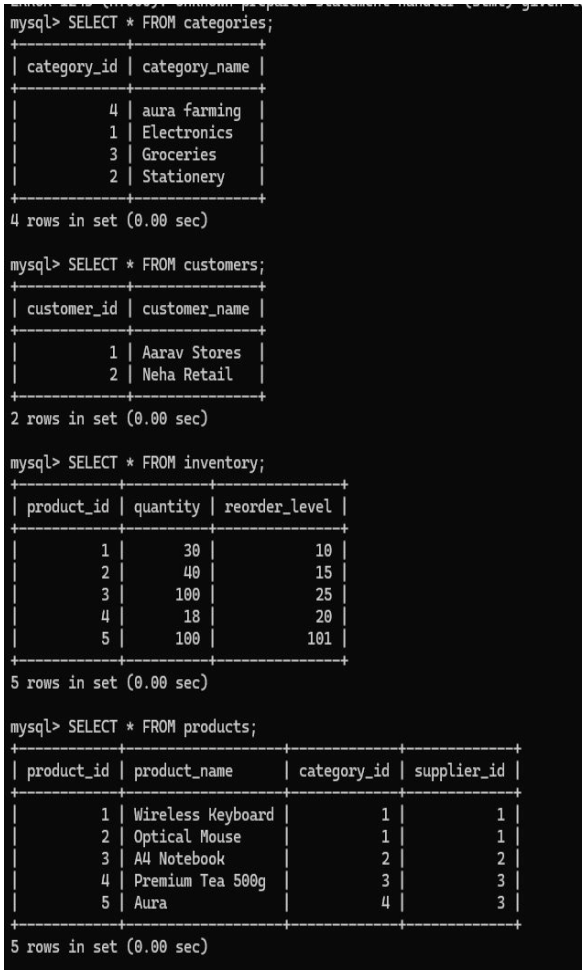


Fig. 6 – Reports Tab: category_inventory_summary VIEW output

Database

The following figures present direct MySQL CLI SELECT * outputs confirming that DatabaseSetup.java correctly created all tables and inserted the sample data. These outputs validate the DDL and DML execution performed through JDBC.

MySQL Output 1: categories, customers, inventory, products



```
mysql> SELECT * FROM categories;
+-------------+---------------+
| category_id | category_name |
+-------------+---------------+
|           4 | aura farming  |
|           1 | Electronics   |
|           3 | Groceries     |
|           2 | Stationery    |
+-------------+---------------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM customers;
+-------------+---------------+
| customer_id | customer_name |
+-------------+---------------+
|           1 | Aarav Stores  |
|           2 | Neha Retail   |
+-------------+---------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM inventory;
+------------+----------+---------------+
| product_id | quantity | reorder_level |
+------------+----------+---------------+
|          1 |       30 |            10 |
|          2 |       40 |            15 |
|          3 |      100 |            25 |
|          4 |       18 |            20 |
|          5 |      100 |           101 |
+------------+----------+---------------+
5 rows in set (0.00 sec)

mysql> SELECT * FROM products;
+------------+-----------------+-------------+-------------+
| product_id | product_name    | category_id | supplier_id |
+------------+-----------------+-------------+-------------+
|          1 | Wireless Keyboard |           1 |           1 |
|          2 | Optical Mouse   |           1 |           1 |
|          3 | A4 Notebook     |           2 |           2 |
|          4 | Premium Tea 500g |           3 |           3 |
|          5 | Aura            |           4 |           3 |
+------------+-----------------+-------------+-------------+
5 rows in set (0.00 sec)
```

*Fig. 7 – MySQL CLI: SELECT * results for categories, customers, inventory, and products tables*

MySQL Output 2: purchase_order_items, purchase_orders, sales_order_items, sales_orders, suppliers

```
mysql> SELECT * FROM purchase_order_items;
+-------------------+------------+----------+
| purchase_order_id | product_id | quantity |
+-------------------+------------+----------+
|                 1 |          1 |        5 |
+-------------------+------------+----------+
1 row in set (0.00 sec)

mysql> SELECT * FROM purchase_orders;
+-------------------+-------------+
| purchase_order_id | supplier_id |
+-------------------+-------------+
|                 1 |           1 |
+-------------------+-------------+
1 row in set (0.00 sec)

mysql> SELECT * FROM sales_order_items;
+----------------+------------+----------+
| sales_order_id | product_id | quantity |
+----------------+------------+----------+
|              1 |          2 |        3 |
+----------------+------------+----------+
1 row in set (0.00 sec)

mysql> SELECT * FROM sales_orders;
+----------------+-------------+
| sales_order_id | customer_id |
+----------------+-------------+
|              1 |           1 |
+----------------+-------------+
1 row in set (0.00 sec)

mysql> SELECT * FROM suppliers;
+-------------+-------------------------+
| supplier_id | supplier_name           |
+-------------+-------------------------+
|           3 | DailyMart Supply        |
|           2 | OfficePro Wholesale     |
|           1 | TechSource Distributors |
+-------------+-------------------------+
3 rows in set (0.00 sec)
```

*Fig. 8 – MySQL CLI: SELECT * results for order and supplier tables*

Output

The complete Warehouse & Inventory Operations Suite successfully executes all JDBC operations across all six modules. The application demonstrates:

#	JDBC Feature	Where Used	Outcome
1	Class.forName() driver loading	Main.java startup	MySQL Connector/J loaded
2	DriverManager.getConnection()	DatabaseManager.java	Connection to javadb established
3	Statement.execute() – DDL	DatabaseSetup.java	9 tables created with FK constraints
4	Statement.execute() – DML	DatabaseSetup.java	Sample data inserted across all tables

5	PreparedStatement INSERT	All Add buttons	Parameterized inserts, SQL-injection safe
6	getGeneratedKeys()	Sales & Purchase Order creation	AUTO_INCREMENT ID retrieved for child insert
7	INSERT … ON DUPLICATE KEY UPDATE	Inventory Save button	Upsert: insert or update in one statement
8	UNION ALL SELECT	Master Data refresh	Three entities displayed in one JTable
9	Multi-table JOIN	Products / Orders listing	Denormalized display of FK-linked records
10	CASE expression in SELECT	Inventory listing	Server-side LOW STOCK / OK computation
11	SQL VIEW query	Reports tab	Aggregated category report via view
12	BEFORE UPDATE Trigger	Inventory update	Negative quantity rejected by DB trigger

All modules operate correctly with live database interaction confirmed through both the Java Swing GUI screenshots and the MySQL CLI output figures presented above.